

# DESIGN AND SYNTHESIS OF LFSR

## Abstract:

This project presents the design, simulation, and synthesis of an LFSR (Linear Feedback Shift Register) is a digital circuit that generates a "pseudo-random" sequence of bits by shifting and recycling data. It works by taking bits from specific positions (called taps), combining them using an XOR gate, and feeding the result back into the first position. This creates a fast, memory-efficient way to scramble data or generate codes for Wi-Fi and encryption.

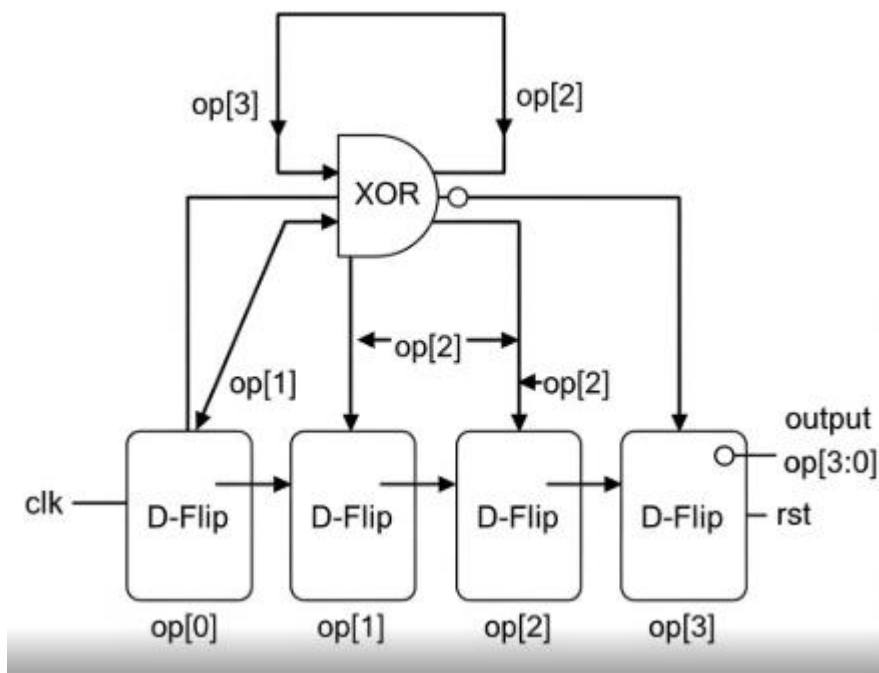
## Introduction:

This project focuses on building a **Linear Feedback Shift Register (LFSR)**, which is a special type of shift register used to generate sequences of numbers that appear random. Unlike a standard counter that goes in order (1, 2, 3...), an LFSR shuffles bits to create a "pseudo-random" pattern.

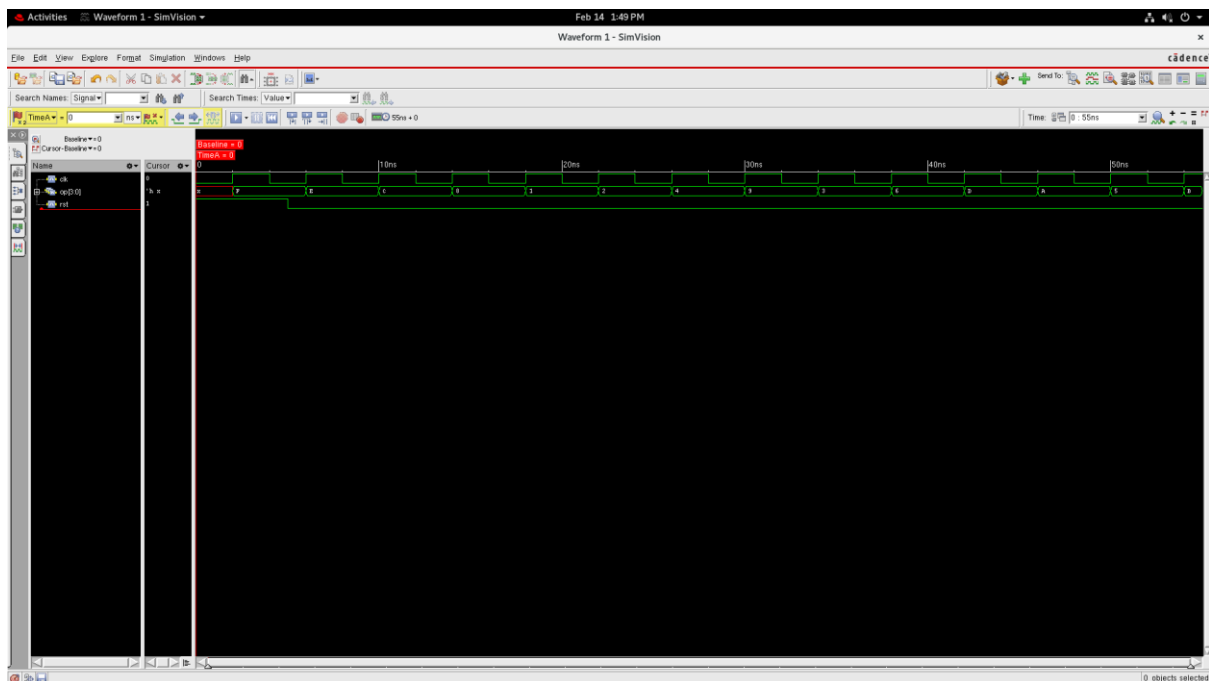
In this project:

- The ALU is designed using Data flow modelling.
- The design is verified through simulation.
- The synthesized netlist is generated using Cadence Genus.
- Timing constraints are applied using an SDC file.

Block Diagram:



### Simulation Results:



## Code:

```
module LFSR(input clk, rst, output reg [3:0] op);
    always@(posedge clk) begin
        if(rst) op <= 4'hf;
        else op = {op[2:0], (op[3]^op[2])};
    end
endmodule
```

## TestBench:

```
module TB;

    reg clk, rst;

    wire [3:0]op;

    LFSR lfsr1(clk, rst, op);

    initial begin

        $monitor("op=%b",op);

        clk = 0; rst = 1;

        #5 rst = 0;

        #50; $finish;

    end

    always #2 clk=~clk;

    initial begin

        $dumpfile("dump.vcd"); $dumpvars;

    end

endmodule
```

## Sdc code:

```
# 1. Define Units (Ensure these match your library, usually ns and pF)
set_units -time 1ns -resistance 1kOhm -capacitance 1pF -voltage 1V -current 1mA

# 2. Create Clock

# Creating a 1GHz clock on the 'clk' port
set clk_period 1.0
```

```
create_clock -name virtual_clk -period $clk_period [get_ports clk]
```

### # 3. Input Delays

# Assuming the 'rst' signal arrives within 20% of the clock cycle

```
set_input_delay -clock virtual_clk -max 0.2 [get_ports rst]
```

### # 4. Output Delays

# Assuming the 'op' bus needs to be stable 20% before the next clock edge

```
set_output_delay -clock virtual_clk -max 0.2 [get_ports op[*]]
```

### # 5. Design Rule Constraints

# Setting a maximum transition time and load to ensure signal integrity

```
set_max_transition 0.1 [current_design]
```

```
set_load 0.05 [get_ports op[*]]
```

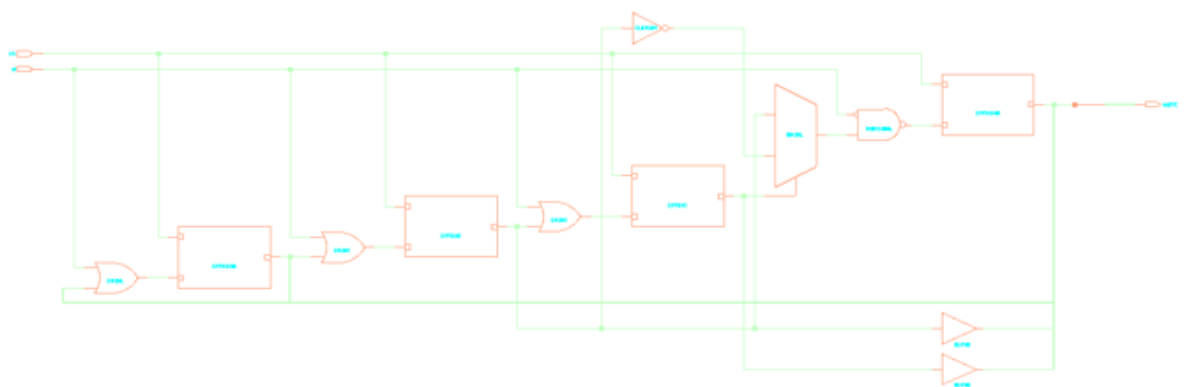
### # 6. Area Constraint (Optional)

# Setting to 0 instructs the tool to make the design as small as possible

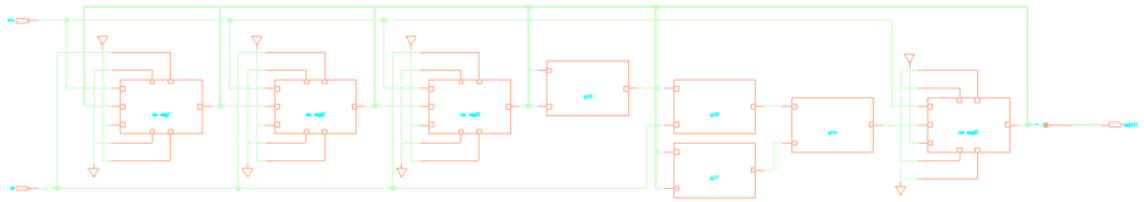
```
set_max_area 0
```

## Synthesis Results:

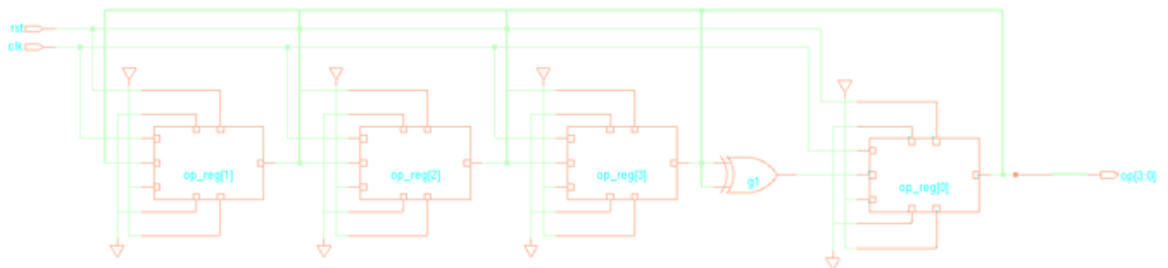
### 1. Elaborated Schematic:



## 2. Generic Schematic:



## 3. Map and Optimized Schematic:



## Reports:

### 1. Power:

| Instance: /lfsr             |              |              |              |              |         |
|-----------------------------|--------------|--------------|--------------|--------------|---------|
| Power Unit: W               |              |              |              |              |         |
| PDB Frames: /stim#0/frame#0 |              |              |              |              |         |
| Category                    | Leakage      | Internal     | Switching    | Total        | Row%    |
| memory                      | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.00%   |
| register                    | 8.01470e-07  | 8.69314e-05  | 8.65343e-06  | 9.63863e-05  | 82.54%  |
| latch                       | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.00%   |
| logic                       | 4.34701e-07  | 8.09544e-06  | 6.51902e-06  | 1.50492e-05  | 12.89%  |
| bbox                        | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.00%   |
| clock                       | 0.000000e+00 | 0.000000e+00 | 5.34600e-06  | 5.34600e-06  | 4.58%   |
| pad                         | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.00%   |
| pm                          | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.00%   |
| Subtotal                    | 1.23617e-06  | 9.50269e-05  | 2.05185e-05  | 1.16781e-04  | 100.01% |
| Percentage                  | 1.06%        | 81.37%       | 17.57%       | 100.00%      | 100.00% |

## 2. Area:

```
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1
Generated on: Feb 14 2026 04:16:05 am
Module: lfsr
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode: timing library
=====
Instance Module Cell Count Cell Area Net Area Total Area Wireload
-----
lfsr 12 127.159 0.000 127.159 <none> (0)
(0) = wireload is default in technology library
```

## 3. Time:

```
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1
Generated on: Feb 14 2026 04:14:46 am
Module: lfsr
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode: timing library
=====
Path 1: MET (293 ps) Setup Check with Pin op_reg[0]/CK->D
Group: virtual_clk
Startpoint: (R) op_reg[3]/CK
Clock: (R) virtual_clk
Endpoint: (R) op_reg[0]/D
Clock: (R) virtual_clk
Capture Launch
Clock Edge:+ 1000 0
Src Latency:+ 0 0
Net Latency:+ 0 (I) 0 (I)
Arrival:= 1000 0
Setup:- 157
Required Time:= 843
Launch Clock:- 0
Data Path:- 550
Slack:= 293
=====
# Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival Instance
# (ps) (ps) (ps) (ps) Location
#-----
op_reg[3]/CK - - R (arrival) 4 - 0 0 0 (-.-)
op_reg[3]/Q - CK->Q F DFFQX1 2 9.0 87 342 342 (-.-)
g40_5107/Y - S0->Y F MXI2XL 1 1.7 88 139 481 (-.-)
g39_2398/Y - B->Y R NAND2BXL 1 2.3 58 70 550 (-.-)
op_reg[0]/D <<< - - R DFFH0XB 1 - 0 0 550 (-.-)
#-----
```

## 4. Conclusion:

The LFSR was successfully designed using data flow modelling Verilog HDL and verified through simulation. The design was synthesized using Cadence Genus with appropriate timing constraints. The results confirm correct functionality and timing compliance. This project demonstrates the complete RTL-to-synthesis flow used in digital VLSI design.